

Modélisation avancée des Systèmes d'Information

03 - Singleton

Mohamed ZAYANI

2025/2026

Définition du Singleton

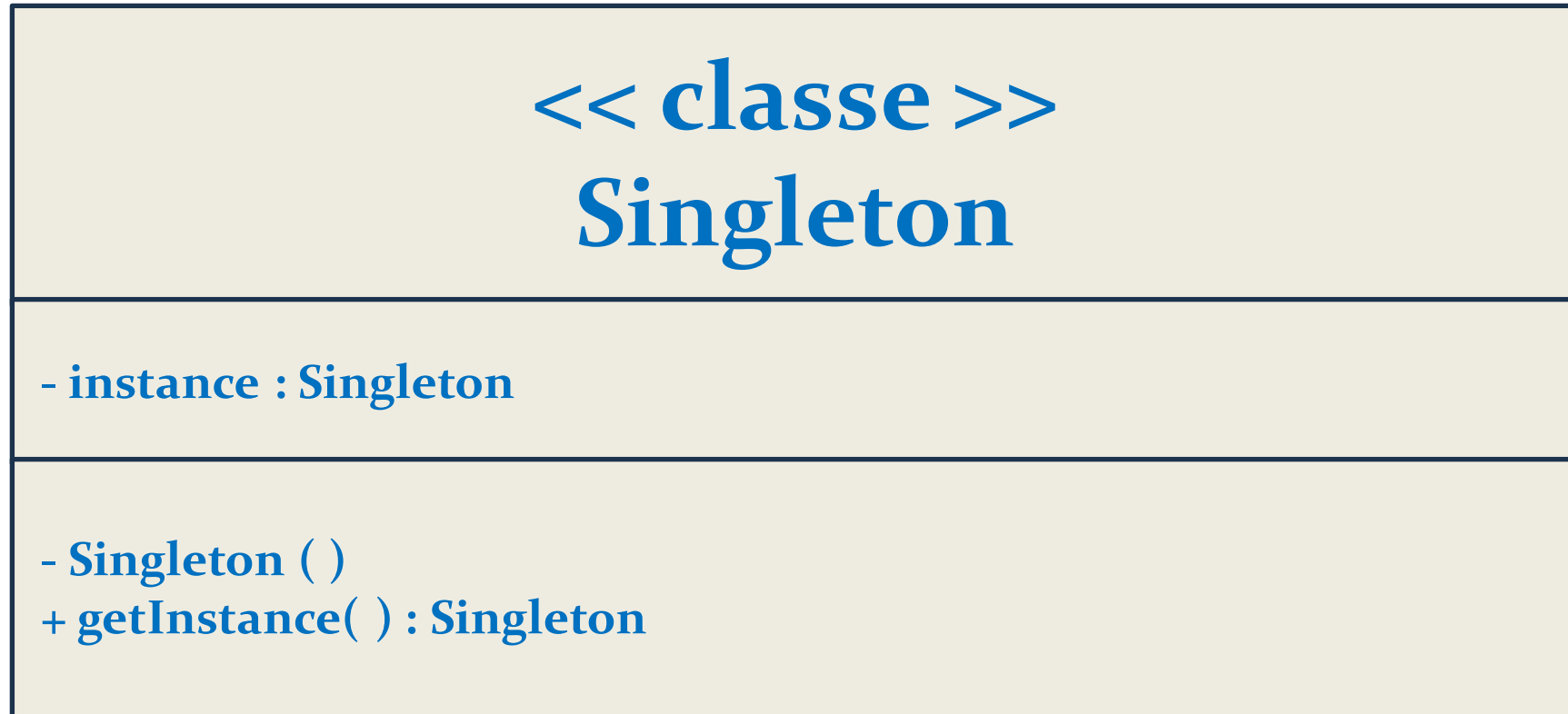
- Le patron de conception Singleton est **un patron créationnel** qui garantit qu'une classe ne possède qu'une seule instance tout en fournissant un point d'accès global à cette instance.
- Objectifs principaux:
 - ❖ Empêcher la création de plusieurs instances d'une même classe,
 - ❖ Fournir un accès contrôlé et unique à l'instance partagée,
 - ❖ Éviter des comportements incohérents liés à des copies multiples

- Problématique:

Dans certaines situations, il est nécessaire d'avoir une seule instance d'une classe pour centraliser des ressources ou des actions, comme :

- Gestion d'une connexion à une base de données.
 - Configuration globale d'une application.
 - Gestionnaire de journaux (logging)..
- Solution:
 - Restreindre la création de nouvelles instances en utilisant **un constructeur privé**.
 - Fournir un point d'accès unique via **une méthode statique** qui retourne toujours la même instance.

Modélisation UML



Implémentation JAVA

```
public class Singleton {
// Instance unique statique
private static Singleton instance;

// Constructeur privé pour empêcher l'instanciation externe
private Singleton() {
System.out.println("Singleton instance créée.");
}

// Méthode statique pour obtenir l'instance unique
public static Singleton getInstance() {
if (instance == null) {
instance = new Singleton();
}
return instance;
}

// Méthode d'exemple pour démonstration
public void afficherMessage() {
System.out.println("Bonjour depuis Singleton !");
}
}
```

```
public class Test {
public static void main(String[] args)
{
// Obtenir l'instance unique
Singleton singleton1 =
Singleton.getInstance();
singleton1.afficherMessage();

// Vérifier que l'instance est unique
Singleton singleton2 =
Singleton.getInstance();
System.out.println(singleton1 == singleton2);
// Affiche true
}
}
```

Résultat:

```
Singleton instance créée.
Bonjour depuis Singleton !
true
```

Gestion de la Concurrency

Dans un environnement **multi-threadé**, plusieurs threads pourraient créer des instances simultanément.

```
package singleton;
public class Test {
    public static void main(String[] args)
    {
        // Thread 1
        Thread thread1 = new Thread(() -> {
            Singleton singleton1 = Singleton.getInstance();
            System.out.println("Instance depuis Thread 1 : " + singleton1);
        });
        // Thread 2
        Thread thread2 = new Thread(() -> {
            Singleton singleton2 = Singleton.getInstance();
            System.out.println("Instance depuis Thread 2 : " + singleton2);
        });
        // Lancer les threads
        thread1.start();
        thread2.start();
    }
}
```

Résultat:

```
Singleton instance créée.
Singleton instance créée.
Instance depuis Thread 2 : singleton.Singleton@29f2d3a4
Instance depuis Thread 1 : singleton.Singleton@748a7303
```

Gestion de la Concurrency (2)

On peut résoudre cela en utilisant la synchronisation.

```
public class Singleton {  
    private static Singleton instance;  
  
    private Singleton() {}  
  
    public static synchronized Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Gestion de la Concurrency (3)

```
package concurrence;
public class Test {
    public static void main(String[] args)
    {
        // Thread 1
        Thread thread1 = new Thread(() -> {
            Singleton singleton1 = Singleton.getInstance();
            System.out.println("Instance depuis Thread 1 : " + singleton1);
        });
        // Thread 2
        Thread thread2 = new Thread(() -> {
            Singleton singleton2 = Singleton.getInstance();
            System.out.println("Instance depuis Thread 2 : " + singleton2);
        });
        // Lancer les threads
        thread1.start();
        thread2.start();
    }
}
```

Résultat:

```
Instance depuis Thread 1 : concurrence.Singleton@47e912d9
Instance depuis Thread 2 : concurrence.Singleton@47e912d9
```

Cependant, la synchronisation sur chaque appel de la méthode peut être coûteuse en performance.

Singleton avec Initialisation à la Demande

Une solution améliorée est l'initialisation à la demande.

```
public class Singleton {  
    private Singleton()  
    {  
        System.out.println("Instance créée.");  
    }  
    private static final Singleton INSTANCE = new Singleton();  
  
    public static Singleton getInstance()  
    {  
        return INSTANCE;  
    }  
}
```

- Cette méthode est à la fois thread-safe et performante.
- L'instance est créée uniquement au moment de l'initialisation du membre statique.